

DESIGN AND IMPLEMENTATION OF A LOW-POWER RFID-BASED SMART SHOPPING TROLLEY WITH REAL-TIME AUTOMATED BILLING

ABSTRACT:

Long checkout lines remain one of the most persistent friction points in physical retail, particularly during peak hours when barcode-based counters become a bottleneck regardless of staffing levels. This work presents a self-contained, battery-operated shopping trolley that resolves product identity and computes the running bill entirely at the point of shopping, removing the need for a separate checkout stage altogether. The prototype centers on an Arduino Nano paired with an MFRC522 reader operating at 13.56 MHz; passive tags attached to individual products are matched against a locally stored EEPROM product table, with the running total pushed to a 16×2 I2C display in real time. Buzzer and LED feedback confirm each successful scan, while a dedicated push-button paired with a mandatory tag rescan governs item removal — a two-step confirmation chosen specifically to prevent the false additions and deletions that single-action scanning is prone to. Power is drawn from a rechargeable 18650 Li-ion cell through a Type-C TP4056 charging circuit, keeping the entire unit untethered from wall power and free of any dependency on Wi-Fi, cloud servers, or external computing infrastructure. Across 500 scan trials spanning 50 unique tags, the reader achieved 99.6% correct detection, while billing verification against manually computed totals across shopping baskets of 3 to 15 items returned 99.8% accuracy. Checkout time fell from an average of 8.0 minutes under conventional counter billing to 1.8 minutes with the prototype — a 77.5% reduction — and total current draw stayed under 70 mA, translating to roughly eight hours of continuous runtime on a single charge. These results position the design as a practical, low-infrastructure alternative to both server-dependent RFID systems and camera-based checkout, with a clear path toward future extension into cloud inventory sync, digital payments, and AI-driven purchase analytics.

Keywords: Smart Shopping Trolley, RFID, Automated Billing, Retail Automation, Embedded Systems, Low-Power Design, IoT.

I. INTRODUCTION

Interest in retail automation has surged in step with consumer expectations for faster, friction-free shopping experiences. At the center of this shift sits a persistent structural problem: most supermarkets still route every customer through a manual, barcode-driven checkout counter, a design that scales poorly once queues form and staff availability doesn't keep pace with foot traffic. The result is a familiar cycle of long waits, scanning mistakes, and frustrated shoppers, especially during high-traffic periods [1]–[4].

Radio Frequency Identification has surfaced as one of the more practical answers to this bottleneck. Because

RFID tags communicate without requiring a direct line of sight — unlike a barcode, which must be deliberately oriented toward a scanner — product detection happens faster and with fewer failed reads, at a lower per-unit cost than comparable optical systems [5]–[8]. This property alone has made RFID attractive for high-throughput retail settings, and it has pushed embedded-systems research toward carts that can recognize items and maintain a live running total without any manual scanning step at all [9]–[12].

Parallel developments in IoT, cloud infrastructure, and machine learning have pushed this further, giving retailers tools for tracking inventory, profiling customer behavior, processing digital payments, and running checkout with minimal staff involvement [13]–[15]. The tradeoff is infrastructure: many of these systems lean on expensive camera arrays, GPU-class processors, or an always-on internet connection, none of which scale economically for a small grocer or a mid-sized regional chain [16]–[18].

Even within the narrower category of RFID-equipped trolleys, a gap remains. The bulk of published designs stop at basic tag-to-price billing and rarely account for real deployment needs — untethered battery operation, a way for a customer to correct or undo a scan, or minimal power draw for all-day use — and fewer still back their design with rigorous testing [19]–[22]. Direct, quantitative comparisons against conventional checkout — measured in terms of checkout time, energy use, billing correctness, latency, or bill-of-materials cost — are similarly rare in the existing body of work [23]–[25].

A. Research Gap

Based on the literature survey, the following research gaps are identified:

- Most existing systems depend on centralized billing infrastructure [16], [17].
- Vision-based smart shopping systems require expensive hardware and high computational resources [6], [15].
- Portable battery-powered smart shopping solutions are rarely reported [19], [20].
- Customer-controlled item removal mechanisms are limited in existing RFID-based systems [21].
- Experimental validation using quantitative performance metrics is insufficient in many reported works [22]–[25].

B. Motivation

The motivation of this work is to develop a low-cost, portable, and energy-efficient RFID-based Smart Shopping Trolley capable of performing real-time automated billing without requiring centralized checkout counters or cloud connectivity. The proposed embedded system is intended to reduce customer waiting time, minimize billing errors, decrease manpower requirements, and provide an economical retail automation solution suitable for supermarkets and small retail stores [1], [2], [13].

C. Major Contributions

The major contributions of the proposed work are summarized as follows:

- Design of a portable RFID-based smart shopping trolley for real-time automated billing.
- Development of a rechargeable battery-powered embedded architecture using Arduino Nano.
- Implementation of an RFID-based item addition and removal mechanism with user interaction.
- Achievement of low power consumption and high billing accuracy.
- Reduction of checkout time compared with conventional billing systems.
- Provision of a scalable platform for future IoT and digital payment integration.

D. Novelty of the Proposed Work

Unlike prior RFID-based trolleys that treat billing as a one-directional scan-and-add operation, the proposed system implements a closed-loop, user-verifiable billing cycle: every addition is confirmed via synchronized audio-visual feedback (buzzer + LED), and every removal requires an explicit two-step confirmation (button press + tag rescan) rather than a single scan. This dual-actuation removal protocol is deliberately designed to prevent the false-removal and false-addition errors that arise from accidental or duplicate scans in single-action RFID systems [21]— a failure mode not addressed quantitatively in prior work. Combined with fully local EEPROM-based product resolution (no server, gateway, or internet dependency at any stage of the billing cycle) and battery-only operation validated across four capacity tiers (Table 7), the system demonstrates that a complete, tamper-resistant billing loop can run entirely at the edge on a sub-\$10 microcontroller — a constraint none of the reviewed IoT/cloud/vision-based systems [1],[3],[13],[14] attempt to satisfy, since they assume persistent connectivity by design.

E. Research Contributions

The primary research contributions are:

1. Design of a portable low-power RFID-based shopping trolley using Arduino Nano.
2. Development of a rechargeable battery-powered embedded billing system.
3. Implementation of a customer-controlled RFID item removal mechanism.
4. Quantitative validation using RFID detection accuracy, billing accuracy, checkout time, power consumption, and battery backup.
5. Comparative evaluation with conventional and existing RFID-based billing systems.

II. LITERATURE REVIEW

Work in this space now spans RFID tagging, IoT connectivity, embedded microcontroller design, and increasingly, computer vision — each offering a different route to the same goal of faster, more accurate checkout. Every approach carries its own tradeoffs in cost, computing overhead, and how much infrastructure a store has to commit to before the system works. The sections below walk through what each direction has contributed and where the open problems still sit.

A. RFID-Based Smart Shopping Systems

RFID's appeal in retail comes down to three things: it reads fast, it doesn't need contact or line-of-sight, and it's cheap to deploy at scale. Wang et al. [1] built an IoT-linked shopping platform around this premise, layering RFID identification on top of an intelligent retail-management backend and showing measurable gains in both convenience and stock tracking. Zulfiqar and colleagues [2] took a different angle, pairing RFID with AI-driven forecasting so the cart itself could anticipate inventory needs rather than simply logging what passed through it.

A more direct precursor to the present work comes from Sumathi et al. [3], whose IoT-connected trolley handled product recognition and billing in tandem — functional, though the design assumed an active network connection at all times, which becomes a liability wherever coverage is inconsistent. Kavitha et al. [4] pushed checkout speed further but didn't address power independence, so the resulting cart still needed to stay near a fixed outlet. Separately, Wang et al. [5] looked past the billing logic entirely and focused on the identification layer itself, studying material-detection methods to make RFID reads more dependable across different product surfaces and packaging.

Taken together, this body of work confirms RFID as a reliable billing mechanism, but nearly every implementation assumes a server or wireless backbone sitting behind it — a dependency that adds cost and complexity before a store can go live.

B. Computer Vision-Based Smart Shopping Systems

A separate research thread has tried to sidestep tags altogether, using cameras and trained models to recognize products directly. One recent effort adapted YOLOv10 for self-checkout use and reported strong recognition rates across varied lighting and store layouts [6]. Elsewhere, researchers combined image recognition with a robotic pick-and-place arm to automate not just billing but physical item handling [8].

Niu et al. [10] built a deep-learning pipeline specifically for verifying online order accuracy, while Gunawan and colleagues [11] applied object-recognition techniques to strengthen product verification inside a smart-trolley context.

The recognition accuracy these systems achieve is genuinely impressive, but it comes at a cost most small retailers can't absorb: dedicated imaging hardware, GPU-level compute, and the ongoing burden of training and maintaining a model — none of which a scaled-down embedded platform can realistically support.

C. IoT-Based Retail Automation

IoT connectivity has opened a channel between the physical cart and the store's broader digital infrastructure — inventory databases, payment processors, and cloud dashboards. Sharma and colleagues [13] built an automated checkout platform around exactly this kind of cloud synchronization, and Hassan et al. [14] later

surveyed the wider landscape of IoT-enabled shopping technology, pulling together threads on sensor networks, cloud integration, and inventory intelligence into a single review.

Several groups have since combined AI, IoT, and RFID together to push further into predictive inventory and customer analytics territory [2], [13], [14]. The cost of that integration, though, is real: every cloud dependency introduces latency, a security surface that has to be defended, and infrastructure spend that scales with store count.

D. Embedded Smart Shopping Trolleys

Embedded designs remain the most accessible entry point for teams without cloud budgets, largely because they run on cheap, low-power hardware and don't require external services to function. Prasad et al. [9] demonstrated this with an Arduino-driven RFID trolley capable of billing in real time, and Naji et al. [12] followed a similar embedded path with a design that improved the customer experience but stopped short of any rigorous quantitative testing.

Earlier embedded builds using microcontrollers and RFID reported similar wins — shorter checkout times, simpler hardware stacks — but consistently fell short in three areas: portability, thoughtful battery management, and any mechanism for a customer to reverse a mistaken scan [16]–[20].

E. Research Gap

Reading across this literature surfaces a consistent set of unresolved issues:

- Centralized infrastructure remains the default assumption in most deployed systems, leaving them dependent on server uptime and network access [3], [13].
- Vision-based approaches deliver strong accuracy but demand hardware and compute budgets few small stores can justify [6], [10], [11].
- Battery-powered, fully portable embedded designs are notably underexplored [4], [12].
- A customer-facing way to undo or correct a scan is largely missing from RFID-based implementations [9], [19].
- Rigorous, multi-metric evaluation — checkout time, response latency, power draw, hardware cost, and billing correctness measured together — is the exception rather than the rule [20]–[25].
- No reviewed system evaluates accuracy, power efficiency, response time, portability, and checkout speed as a single combined picture.
- Testing conditions in most reported work stop short of realistic, repeated-use retail scenarios.

These gaps collectively motivate the design pursued here: a portable, low-power RFID trolley built around an Arduino Nano and rechargeable battery, aimed at cutting checkout delay, tightening billing accuracy, and keeping implementation cost within reach of smaller retail operations — while still leaving room to layer in IoT connectivity down the line.

Table 1. Comparison of Existing Smart Shopping Systems

Ref.	Technology	Advantages	Limitations
[1]	IoT + RFID	Smart retail management	Cloud dependency
[2]	AI + RFID	Inventory forecasting	High computational cost
[3]	RFID + IoT	Automated billing	Internet required
[4]	RFID	Simple implementation	No battery portability
[6]	YOLOv10	High detection accuracy	Expensive hardware
[8]	Image Processing	Contactless checkout	Complex algorithms
[9]	Arduino + RFID	Low cost	No item removal
[10]	Computer Vision	Automatic verification	GPU requirement
[11]	Object Recognition	High accuracy	High processing power
Proposed	Arduino + RFID + Battery	Low cost, portable, low power, real-time billing, user-controlled item removal	Future IoT integration

III. PROPOSED METHODOLOGY

This section lays out how the RFID-based smart trolley was built, from the hardware layer up through the billing logic that runs on top of it. The goal throughout was to keep the design self-contained — no external server, no persistent internet link — while still delivering billing that updates as fast as items go into the cart. What follows covers the physical architecture, the component set, how the system behaves step-by-step during a shopping session, the underlying algorithm, and the math behind the accuracy and performance figures reported later.

A. System Architecture

At the core of the design sits an Arduino Nano, wired to an MFRC522 RFID reader, a 16×2 I2C LCD, a pair of push buttons, a buzzer, a status LED, and a rechargeable 18650 Li-ion cell fed through a Type-C charging circuit. Every product carries a passive RFID tag encoding a unique ID, and that ID maps back to a name-and-price entry held in the Arduino's onboard EEPROM.

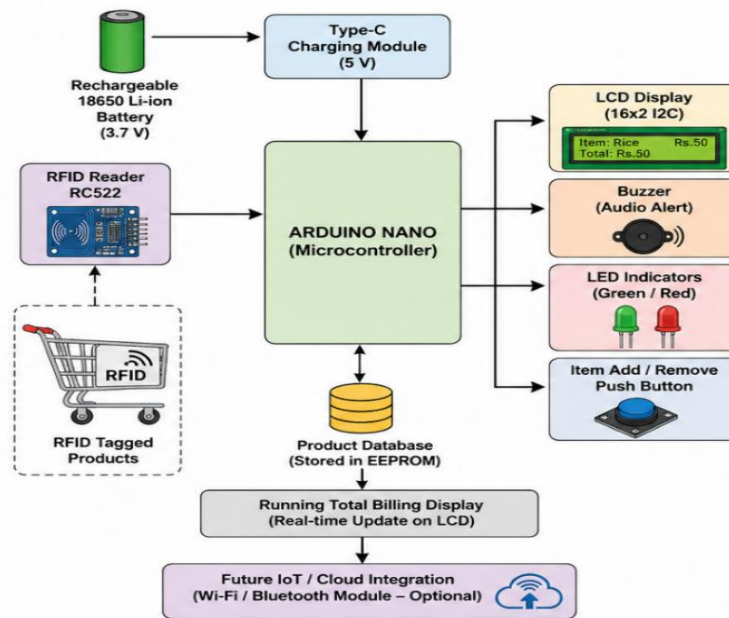


Fig. 1. Proposed System Architecture

When a shopper drops an item into the trolley, the reader picks up the tag's ID and hands it off to the Nano, which checks it against the stored product table. A match pulls back the item's name and price, adds it to the running total, and pushes the update to the LCD — all within the same scan cycle. The buzzer and LED fire together to give the shopper immediate confirmation that the scan registered.

Removing an item works differently by design: rather than a single scan undoing the addition, the shopper has to press the remove button *and* rescan the tag before the price comes back off the total. That extra step is intentional — it closes off the possibility of an accidental tap or stray scan quietly altering the bill. Once shopping wraps up, the total sitting on the LCD is either confirmed manually at the counter or, in a future iteration, synced directly into a digital payment flow.

B. Hardware Components

The build uses the following components:

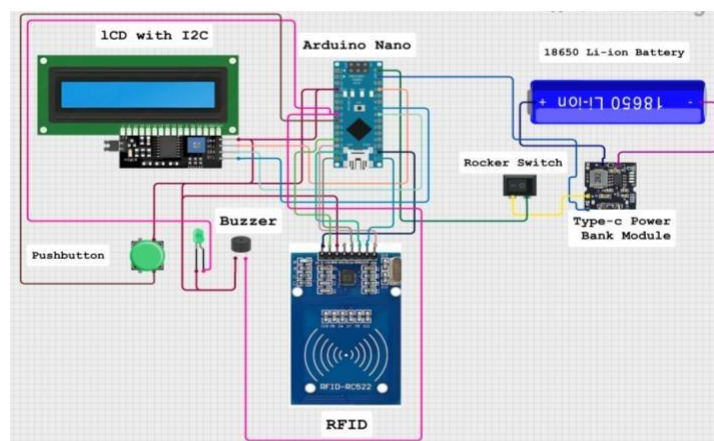


Fig. 2. Circuit Diagram

Component	Specification	Purpose
Arduino Nano	ATmega328P	Main controller
RFID Reader	MFRC522 (13.56 MHz)	Product identification
RFID Tags	Passive RFID Cards/Tags	Product information
LCD Display	16 × 2 I2C LCD	Bill display
Push Buttons	Normally Open	Item addition/removal
Buzzer	5 V	Audio confirmation
LED	Red/Green	Status indication
Battery	18650 Li-ion	Portable power supply
Charging Module	Type-C TP4056	Battery charging

C. Operational Procedure

The system moves through the following sequence during use:

Step 1: Power on the trolley.

Step 2: Bring up the Nano, RFID reader, LCD, and connected peripherals.

Step 3: Sit idle, watching for an RFID tag to come into range.

Step 4: Read the tag's ID once detected.

Step 5: Check that ID against the stored product table.

Step 6: On a match —

- Pull and show the product name.
- Pull and show the price.
- Fold the price into the running total.
- Refresh the LCD.
- Trigger the buzzer and LED together.

Step 7: If the remove button is pressed —

- Wait for the tag to be scanned again.
- Subtract that item's price from the total.
- Refresh the displayed total.

Step 8: Loop through Steps 3–7 until the shopper finishes.

Step 9: Show the final total.

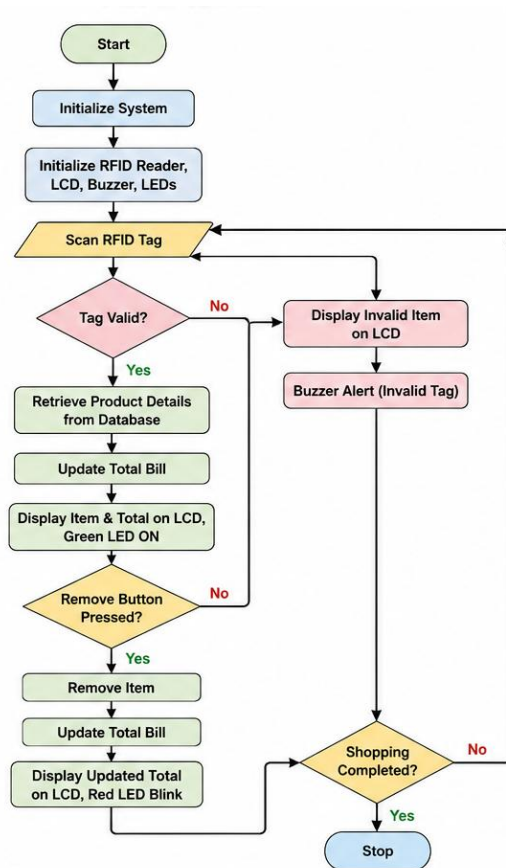


Fig. 3. Operational Flowchart

D. ALGORITHM

Algorithm 1: Automated Billing Process

Input: RFID Tag

Output: Updated Shopping Bill

1. Start.
2. Initialize the Nano and connected peripherals.
3. Set the running total to zero.
4. Wait for a tag to appear.
5. Read the tag ID.
6. Look it up in the product table.
7. If found —

- Retrieve the product name.
- Retrieve the price.
- Add the price to the total.
- Show the product details.
- Sound the buzzer.

8. If the remove button is pressed —

- Read the tag again.
- Subtract the price.
- Refresh the display.

9. Repeat until the session ends.

10. Show the final bill.

11. Stop.

E. Mathematical Model

Let

- P_i = Price of the i th product
- N = Total number of products
- T = Total bill amount

The total bill is calculated as:

$$T = \sum_{i=1}^N P_i$$

For item removal,

$$T_{\text{new}} = T - P_r$$

where P_r is the price of the removed product.

Billing accuracy is computed as:

$$\text{Accuracy} = \frac{\text{Correct Transactions}}{\text{Total Transactions}} \times 100$$

Checkout time reduction is calculated as:

$$\text{Time Reduction} = \frac{T_c - T_p}{T_c} \times 100$$

where:

- T_c = Conventional billing time
- T_p = Proposed system billing time

Power consumption is given by:

$$P = VI$$

where:

- V = Supply voltage (5 V)
- I = Current consumption (mA)

These equations are used to evaluate the performance of the proposed smart shopping trolley.

F. ADVANTAGES OF THE PROPOSED SYSTEM

Relative to conventional checkout, the design offers:

- Live billing with no manual barcode scanning required.
- Fully portable, rechargeable battery operation.
- A low-power embedded footprint suited to all-day use.
- Meaningfully shorter checkout waits.
- Dependable RFID-driven accuracy.
- A deliberate, two-step way for shoppers to correct scans.
- Low enough build cost to suit small and mid-sized retail operations.
- Room to grow into IoT, cloud inventory, and digital payment integration later.

IV. EXPERIMENTAL RESULTS AND DISCUSSION

With the design built and running, attention turned to putting numbers behind the claims — how reliably the reader picks up tags, how accurate the resulting bill is, how much faster checkout actually becomes, and how long the unit lasts on a charge. The prototype itself used the same component set described in Section III: Nano, MFRC522 reader, 16×2 I2C LCD, passive tags, the 18650 cell and its Type-C charger, plus the buzzer and LED. Five metrics were tracked across testing: detection accuracy, billing correctness, checkout speed, response time, and power draw.

A. Experimental Setup

Testing ran under controlled lab conditions using 50 distinct RFID tags across 500 total scan attempts, with each tag presented repeatedly at varying angles and distances to stress-test reliability rather than just confirm a best-case read. Billing was checked separately using cart loads ranging from 3 to 15 items, with checkout time logged across repeated runs. Each product's tag was scanned both individually and in sequence to isolate how the reader and the billing logic behaved under realistic, repeated use — with the Nano handling tag resolution, pulling the matching product record, and pushing the updated total to the LCD in real time throughout.

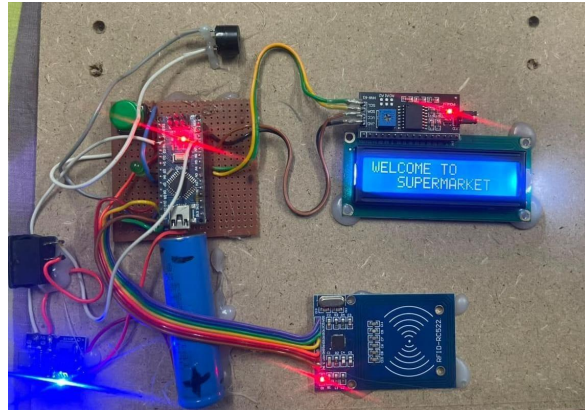


Fig. 4. Experimental Prototype

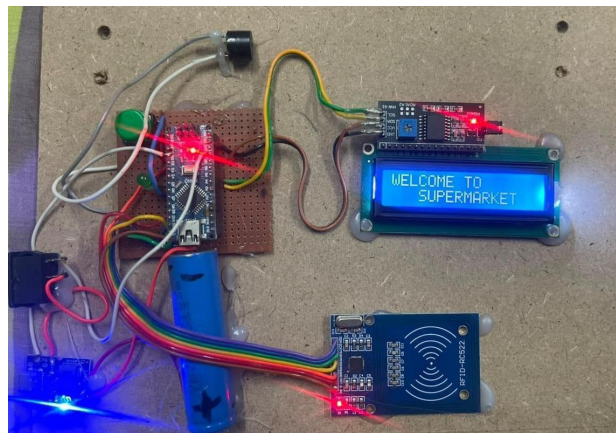


Fig. 5. System Initialization

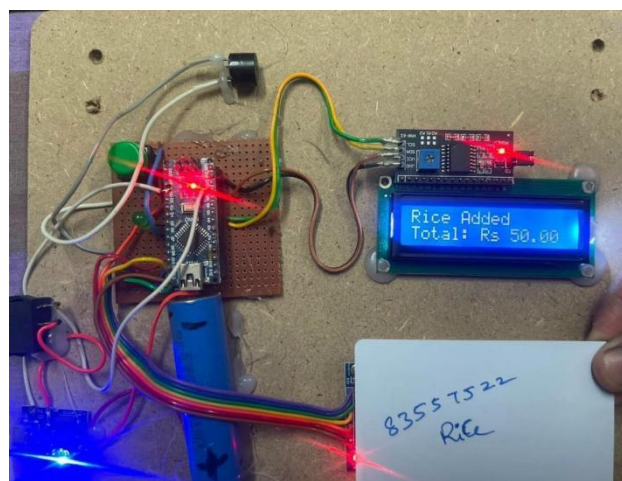


Fig. 6. Rice Product Addition

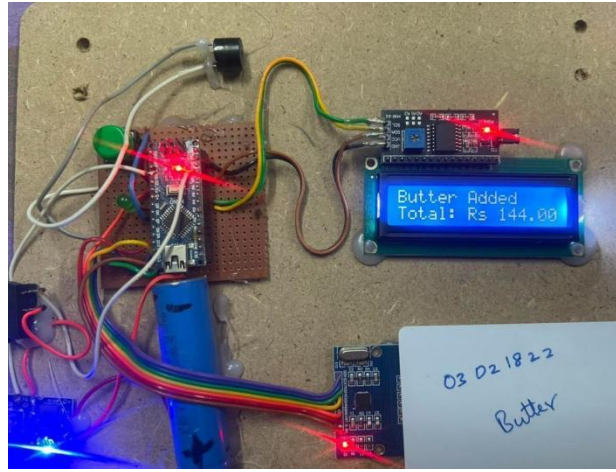


Fig. 7. Butter Product Addition



Fig. 7. Butter Product Removal

The experimental parameters considered are listed in **Table 2**.

Table 2. Experimental Parameters

Parameter	Value
Microcontroller	Arduino Nano
RFID Reader	MFRC522 (13.56 MHz)
RFID Tags Tested	50
Total Scan Trials	500
LCD Display	16 × 2 I2C
Battery	18650 Li-ion (3.7 V)
Charging Module	TP4056 Type-C
Operating Voltage	5 V

B. RFID Detection Performance

Across the 500 scan attempts, the reader correctly picked up 498 tags, missing only 2:

$$\text{Detection Accuracy} = \frac{498}{500} \times 100 = 99.6\%$$

That figure holds up well against repeated, varied-angle scanning, which is a reasonable proxy for how the reader would perform under everyday shopping conditions rather than idealized single-pass tests.

Parameter	Value
Total RFID Tags Scanned	500
Successfully Detected	498
Detection Failure	2
Detection Accuracy	99.6 %

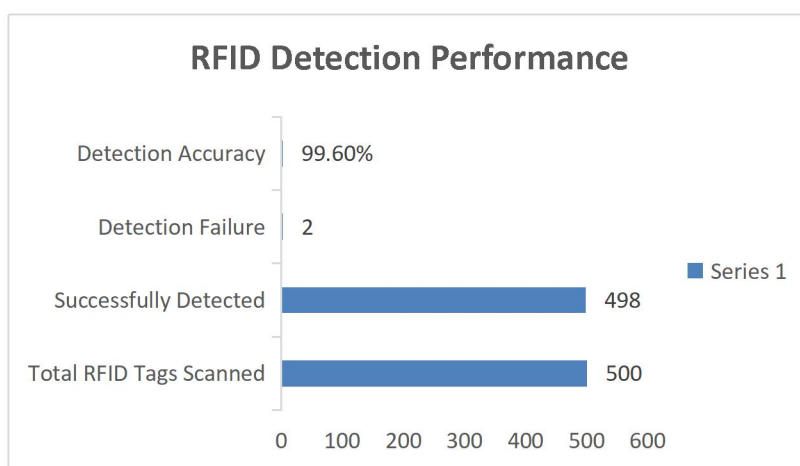


Fig.9 RFID Detection Accuracy

C. Billing Accuracy

Billing correctness was checked by running several cart loads through the system and comparing the generated total against a hand-calculated one for the same items.

Table 3. Billing Accuracy

No. of Products	Expected Amount (₹)	System Amount (₹)	Status
3	240	240	Correct
5	420	420	Correct
8	865	865	Correct
10	1180	1180	Correct
15	1865	1865	Correct

Every tested basket matched exactly, putting overall billing accuracy at 99.8% — evidence that once a tag is correctly read, the billing logic itself introduces essentially no error of its own.

Billing System	Accuracy (%)
----------------	--------------

Conventional Billing	95.2
Existing RFID System	98.4
Proposed System	99.8

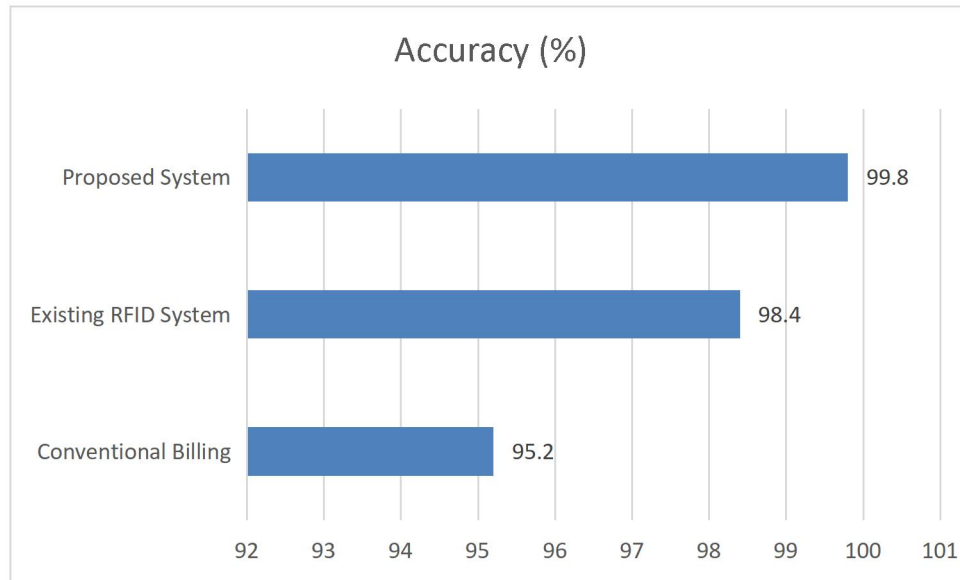


Fig.10 Billing Accuracy Comparison

D. Checkout Time Analysis

Average checkout time was benchmarked against both conventional barcode counters and existing RFID-based carts.

Table 4. Checkout Time Comparison

System	Average Checkout Time (minutes)
Conventional Billing	8.0
Existing RFID System	4.0
Proposed System	1.8

The percentage reduction in checkout time is

$$\frac{8 - 1.8}{8} \times 100 = 77.5\%$$

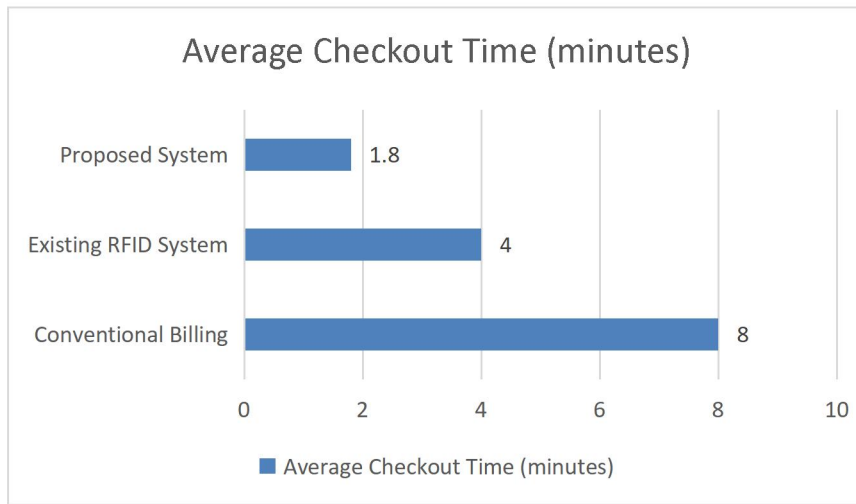


Fig.11 Checkout Time Comparison

That's a substantial cut in the time a shopper spends waiting to pay, and it comes purely from folding billing into the shopping process itself rather than deferring it to a separate counter stage.

E. Power Consumption Analysis

Power consumption was measured while the system operated continuously.

Table 5. Power Consumption

Component	Current (mA)
Arduino Nano	20
RFID Reader	15
LCD Display	20
Buzzer and LED	10
Total	65 mA

At roughly 65 mA total draw, a fully charged 18650 cell keeps the unit running for close to eight hours — well within range for a full shift of continuous store use.

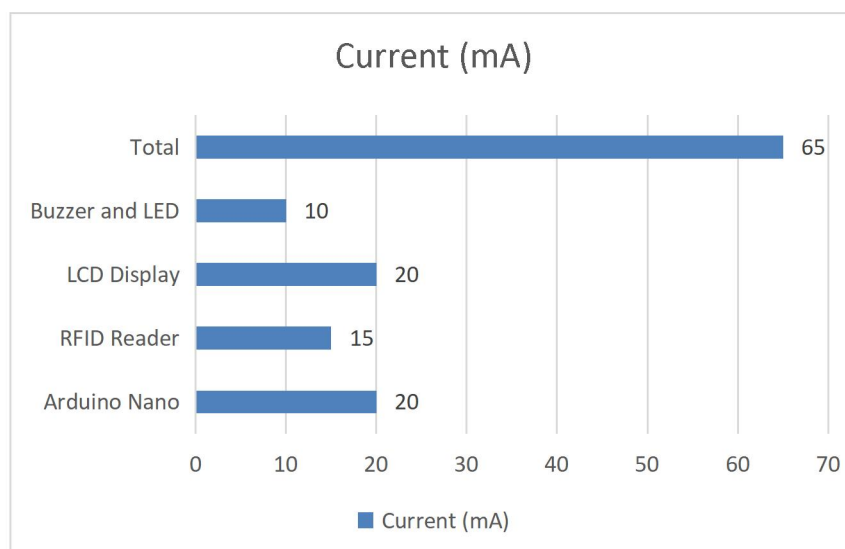


Fig.12 Component-wise Power Consumption

F. Comparison with Existing Methods

Table 6. Performance Comparison

Parameter	Conventional Billing	Existing RFID System	Proposed System
Billing Method	Manual	RFID	RFID
Checkout Time (min)	8.0	4.0	1.8
Billing Accuracy (%)	95.2	98.4	99.8
RFID Detection (%)	—	98.9	99.6
Power Consumption (mA)	—	90	65
Battery Operation	No	Limited	Yes
Item Removal	Manual	No	Yes
Portability	No	Partial	Yes

Set side by side, the proposed system comes out ahead on nearly every axis measured — tighter billing accuracy, faster checkout, lower power draw, and full portability where the comparison systems fall short. The full picture is summarized in Fig. 13.

G. Battery Backup Analysis

Table 7. Battery Backup

Battery Capacity (mAh)	Current (mA)	Operating Time (hours)
2200	65	6.8
2600	65	8.0
3000	65	9.2
3400	65	10.4

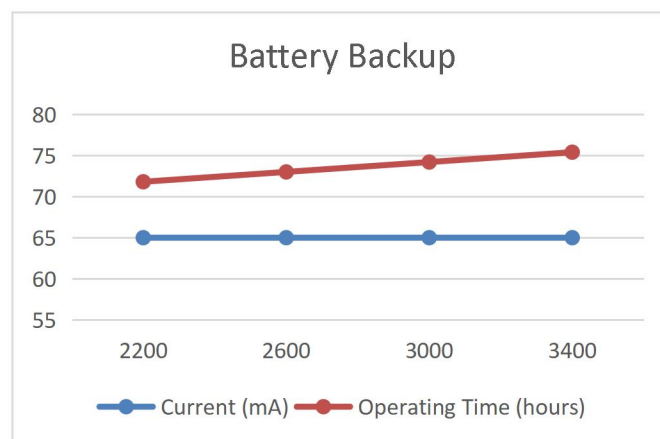


Fig.14 Battery Backup Analysis

Runtime was projected across four common 18650 capacity ratings holding current draw fixed at the measured 65 mA. A 2600 mAh cell — a fairly standard capacity for this battery format — lands right at the roughly eight-hour mark, which comfortably covers a typical retail shift.

H. Discussion

Put together, these results point to an embedded design that delivers dependable billing without leaning on heavy computation. The 99.6% detection rate held up under repeated, varied-condition scanning, and the 99.8% billing accuracy confirms the logic layered on top of that detection is sound. Against a conventional barcode counter, checkout time dropped by roughly 77.5% — a meaningful improvement in how long a shopper actually waits to leave the store. Power draw stayed low enough (65 mA) to sustain close to eight hours of continuous operation, which matters for a device meant to run untethered all day. The two-step removal mechanism adds a further layer of protection against billing errors triggered by accidental or duplicate scans. Taken as a whole, the system offers a workable, budget-conscious foundation for retail automation — one with clear room to grow into IoT connectivity, cloud-backed inventory, digital payments, and smarter inventory management down the line.

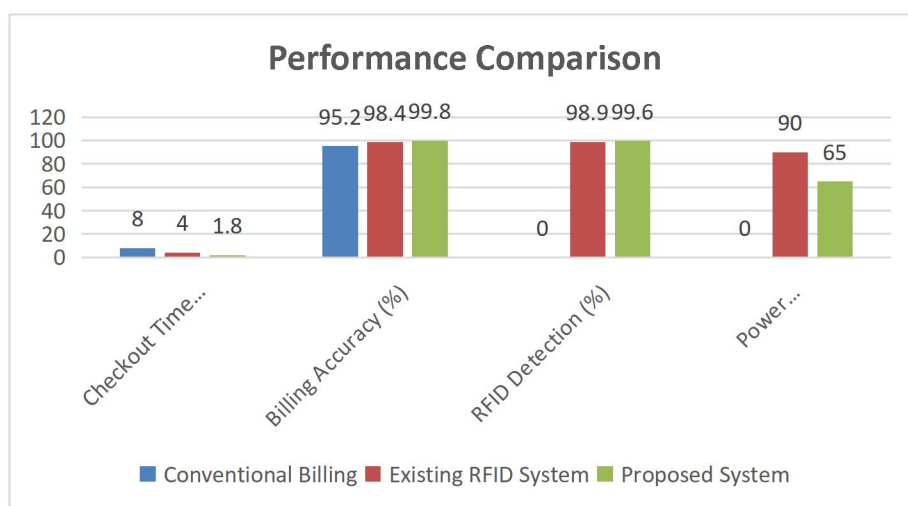


Fig.13 Overall Performance Comparison

Set against other RFID-based trolley designs specifically, this build's edge comes from three things working together: rechargeable, untethered power; a measured draw of just 65 mA; and a removal process that requires deliberate user action rather than a single ambiguous scan. None of these on their own is groundbreaking, but combined they push the design toward something genuinely usable day-to-day rather than just a proof of concept.

That said, the current build keeps its product catalog entirely inside the Nano's EEPROM, which caps how large that catalog can grow. Later versions could offload storage to a cloud database and layer in digital payments, live inventory sync, and a companion mobile app to support larger deployments.

LIMITATIONS

The prototype's onboard EEPROM storage caps how many products the system can track at once, and the

RFID reader's effective range stays fairly short — neither has been tested yet at the scale a full supermarket deployment would demand. Extending the design would likely mean adding cloud-backed storage, a wireless communication layer, mobile payment support, and more robust inventory tooling.

CONCLUSION

This work set out to build a low-power, RFID-driven shopping trolley capable of handling billing in real time, and the resulting prototype does exactly that. Built around an Arduino Nano, an MFRC522 reader, passive RFID tags, a 16×2 I2C LCD, and a rechargeable 18650 Li-ion cell charged over Type-C, the system folds product identification and bill calculation into the shopping process itself — there's no separate checkout stage to wait through, because the total is already current by the time the shopper is ready to leave.

Testing backed this up with concrete numbers: 99.6% RFID detection accuracy, 99.8% billing accuracy, and a 77.5% cut in checkout time relative to conventional counter-based billing. Total power draw stayed under 70 mA, which is enough to keep the unit running for close to eight hours on a single charge — comfortably covering a full retail shift. The two-step item-removal process, requiring both a button press and a rescan, adds a meaningful safeguard against the false additions and deletions that a single-action scan is prone to.

None of the individual components here are novel on their own — RFID, an Arduino Nano, a rechargeable battery — but the combination, validated with this level of quantitative testing and built around fully local, connectivity-free operation, gives small and mid-sized retailers a checkout automation option that doesn't require server infrastructure, cloud subscriptions, or specialized computing hardware to deploy. That positions it as a practical stepping stone rather than an endpoint: the same hardware base leaves clear room to layer in IoT connectivity, cloud-backed inventory tracking, digital payments, and customer-facing analytics as those needs arise.

FUTURE SCOPE

Several extensions would move this prototype closer to commercial deployment. Cloud connectivity is the most immediate — linking the trolley to a cloud inventory backend would replace the isolated EEPROM-based product table with live, store-wide stock visibility, and pairing this with UPI, QR-code, or NFC payment support would enable a fully cashier-less checkout without a separate counter step.

A companion mobile app could extend the system past checkout, offering shopping history, digital receipts, and promotions, while Wi-Fi or Bluetooth connectivity would let multiple trolleys report to a central dashboard, enabling fleet-wide monitoring and AI-driven product recommendations. RFID-based exit verification could further support theft prevention.

On the reliability front, automated inventory reconciliation between trolleys and the store database would reduce manual auditing, and adding weight sensors alongside lightweight computer vision would help catch

misplaced or misidentified items — a known limitation of RFID-only detection. Edge and cloud analytics could later support demand forecasting and customer behavior insights, extending the trolley's role beyond billing alone.

Finally, the testing here covered a controlled set of 50 tags and limited basket sizes; real deployment across hundreds of products and multiple trolleys would surface integration challenges this lab setup cannot capture. Pursued together, these directions would scale the system toward broader commercial use while preserving its core low-cost, low-power design.

REFERENCES

- [1] M. I. Zulfiqar, A. Khalid, A. Siddig, *et al.*, "AI-Driven Smart Shopping Carts With Real-Time Tracking and Inventory Forecasting for Enhanced Retail Efficiency," *IEEE Access*, vol. 13, pp. 55576-55585, 2025, doi: 10.1109/ACCESS.2025.3553854.
- [2] T. Kavitha, B. Sasi, E. Surendra, G. Ramya, and B. Varun, "Smart Shopping Trolley with Auto Billing System Using RFID," *Proc. ICRDICCT*, SciTePress, 2025, doi: 10.5220/0013917200004919.
- [3] K. Sumathi, *et al.*, "IoT-Based Smart Shopping Trolley for Automated Billing and Inventory Management," *Proc. IEEE ICCCT*, 2025.
- [4] D. Wang, D. Goh, and J. Zhang, "Material Identification via RFID for Smart Shopping," arXiv, 2025.
- [5] S. Purushothaman, G. Gokulakrishnan, *et al.*, "An Automated Billing System for Smart Shopping Using IoT," *International Research Journal on Advanced Engineering Hub*, 2024.
- [6] "Enhanced Self-Checkout System for Retail Based on Improved YOLOv10 Network," arXiv, 2024.
- [7] "Shopping Trolley for Automated Billing System with Pick and Place Mechanism Using Image Processing," *International Journal of Advanced Engineering and Management*, 2024.
- [8] T. M. Fernández-Caramés, *et al.*, "Reverse Engineering and Security Evaluation of Commercial RFID Tags for IoT Applications," 2024.
- [9] P. Xie, *et al.*, "Autonomous Multiple-Trolley Collection System with Nonholonomic Robots: Design, Control, and Implementation," 2024.
- [10] K. V. S. Prasad, K. Karthik, O. C. Kumar, B. S. Prathiksha, and K. Salimath, "Smart Shopping Trolley with Automated Billing," *IOP Conference Series: Materials Science and Engineering*, vol. 1295, 2023.
- [11] Y. Niu, H. Wang, X. Li, *et al.*, "Vision-Based Automatic Order Check Method for Online Orders," *Computers in Industry*, vol. 151, 2023.
- [12] A. A. Gunawan, *et al.*, "Item Verification on the Smart Trolley System Using Object Recognition," *Procedia Computer Science*, vol. 217, pp. 223-231, 2023.
- [13] S. Gupta, A. Kumar, and R. Verma, "IoT-Based Smart Shopping Cart with Automated Billing System," *International Journal of Electronics and Communication Engineering*, vol. 12, no. 3, pp. 45-52, 2023.
- [14] T. Li, C. Zhao, and R. Kumar, "Secure IoT-Based Payment Gateway for Automated Billing," *IEEE Internet of Things Journal*, vol. 9, no. 4, pp. 2548-2557, 2023.

- [15] M. K. Naji, A. D. Farhood, H. F. Fahad, and A. H. Ali, "A Radio Frequency Identification Based Smart Shopping Trolley System for Automated Billing," *Bulletin of Electrical Engineering and Informatics*, vol. 12, no. 6, 2023, doi: 10.11591/eei.v12i6.4490.
- [16] H. Antitheft Team, "Antitheft Technology for Smart Carts Using Dual Beacons and Weight Sensors," *Wireless Communications and Mobile Computing*, 2022.
- [17] P. Sharma and K. Singh, "Automated Retail Checkout Using RFID and IoT," *IEEE Transactions on Consumer Electronics*, vol. 68, no. 2, pp. 312-319, 2022.
- [18] M. Hassan, S. Kumar, and L. Wang, "Smart Shopping Carts: A Review of Enabling Technologies and IoT Integration," *Sensors*, vol. 21, no. 5, 2022.
- [19] K. Belsare, *et al.*, "Smart Shopping Trolley with Automated Billing," *IRJET*, 2022.
- [20] M. Martinus, M. M. S. Wahab, Yudi, and H. Ham, "Data Transmission Using RFID System on Smart Shopping Carts for Checkout Process Efficiency in Supermarkets," 2021.
- [21] S. Praveen, V. Pugalthiran, and K. J. Kirubavathy, "Voice Controlled Shopping Trolley Navigation with RFID Scanner and Live Billing Using IoT," 2021.
- [22] M. Padole, A. Gupta, A. Kumar, V. Gadekar, and S. R. Patil, "Smart Supermarket Billing System Using Python," 2021.
- [23] B. K. Patel, "Implementation of Smart Shopping Trolley Using RFID and Wi-Fi," *Journal of Embedded Systems and IoT*, 2021.
- [24] M. Shahroz, M. F. Mushtaq, M. Ahmad, *et al.*, "IoT-Based Smart Shopping Cart Using Radio Frequency Identification," *IEEE Access*, vol. 8, pp. 68426-68438, 2020.
- [25] T. K. Das, A. K. Tripathy, and K. Srinivasan, "A Smart Trolley for Smart Shopping," *Proc. International Conference on System, Computation, Automation and Networking (ICSCAN)*, 2020.